

1. 物理问题描述与设定

考虑处于一个无限深方势阱中的粒子。在势阱内部叠加一高斯形状的扰动势。定态薛定谔方程无法解析求解。

- 势阱范围： $(0 \leq x \leq L)$ 。
- 总势能函数 $V(x)$:

$$V(x) = V_{ISW}(x) + V_{perturbation}(x)$$

其中:

- $V_{ISW}(x)$ 是无限深方势阱的势能:

$$V_{ISW}(x) = \begin{cases} 0 & 0 \leq x \leq L \\ \infty & \text{otherwise} \end{cases}$$

- $V_{perturbation}(x)$ 是一个高斯扰动势:

$$V_{perturbation}(x) = V_0 \exp\left(-\frac{(x - x_c)^2}{2\sigma^2}\right)$$

其中 V_0 是扰动势的强度, x_c 是高斯峰的中心位置, σ 是高斯峰的宽度。

- 定态薛定谔方程: $\hat{H}\psi(x) = E\psi(x)$ 其中 哈密顿算符 $\hat{H} = -\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + V(x)$
- 求解系统的能量本征值 E 和对应的波函数本征函数 $\psi(x)$ 。

2. 物理公式

2.1 基函数展开

将波函数 $\psi(x)$ 在一组完备的正交归一基函数 $\{\phi_n(x)\}$ 上展开。不妨选取基函数为无穷深方势阱的本征函数, 满足边界条件 $\psi(0) = \psi(L) = 0$ 。

无限深方势阱的本征函数为:

$$\phi_n(x) = \sqrt{\frac{2}{L}} \sin\left(\frac{n\pi x}{L}\right), \quad n = 1, 2, 3, \dots$$

将波函数截断到有限项 N_{basis} : $\psi(x) = \sum_{n=1}^{N_{basis}} c_n \phi_n(x)$ 其中 c_n 是展开系数。

2.2 哈密顿量矩阵的构建

将展开式代入薛定谔方程, 并左乘 $\langle \phi_m |$, 积分, 得矩阵形式的本征值问题:

$$\mathbf{H}\mathbf{c} = E\mathbf{c}$$

哈密顿量矩阵 $\mathbf{H}$ 满足

$$H_{mn} = \langle \phi_m | \hat{H} | \phi_n \rangle = \int_0^L \phi_m^*(x) \hat{H} \phi_n(x) dx$$

由于 $\hat{H} = \hat{T} + \hat{V}$, 其中动能算符 $\hat{T} = -\frac{\hbar^2}{2m} \frac{d^2}{dx^2}$, $\hat{V} = V_{\text{perturbation}}(x)$ 是扰动势能算符, 得 $H_{mn} = \langle \phi_m | \hat{T} | \phi_n \rangle + \langle \phi_m | \hat{V}_{\text{perturbation}} | \phi_n \rangle$

- 动能项 $\langle \phi_m | \hat{T} | \phi_n \rangle$: 由于 $\phi_n(x)$ 是无限深方势阱的本征函数, 它们也是动能算符的本征函数:

$$\hat{T} \phi_n(x) = \frac{n^2 \pi^2 \hbar^2}{2mL^2} \phi_n(x) = E_n^0 \phi_n(x)$$

因此, 动能项是:

$$\langle \phi_m | \hat{T} | \phi_n \rangle = E_n^0 \langle \phi_m | \phi_n \rangle = E_n^0 \delta_{mn} = \frac{n^2 \pi^2 \hbar^2}{2mL^2} \delta_{mn}$$

这是动能部分的哈密顿量矩阵, 已自然地被对角化。

- 势能项 $\langle \phi_m | \hat{V}_{\text{perturbation}} | \phi_n \rangle$:

$$\langle \phi_m | \hat{V}_{\text{perturbation}} | \phi_n \rangle = \int_0^L \phi_m(x) V_{\text{perturbation}}(x) \phi_n(x) dx$$

这个积分通常无法解析求解, 需要使用数值积分。

2.3 矩阵对角化

构建出哈密顿量矩阵 $\mathbf{H}$ 后, 我们使用 `numpy.linalg.eigh` 函数对其进行对角化。这将返回:

- 特征值 E : 对应于系统的能量本征值。
- 特征向量 $\mathbf{c}$: 对应于每个能量本征值的展开系数向量。

2.4 波函数的重构

得到特征向量 $\mathbf{c}$ 后, 我们可以根据展开式 $\psi(x) = \sum_{n=1}^{N_{\text{basis}}} c_n \phi_n(x)$ 重构出对应的波函数。

3. 常数值设定

- 物理常数:
 - 约化普朗克常数 $\hbar = 1.0545718 \times 10^{-34}$ J·s
 - 电子质量 $m_e = 9.1093837 \times 10^{-31}$ kg
- 势阱参数:
 - 势阱宽度 $L = 1.0 \times 10^{-9}$ m
- 扰动势参数:
 - 扰动强度 $V_0 = 5.0 \times 1.602 \times 10^{-19}$ J
 - 扰动中心 $x_c = L/2$
 - 扰动宽度 $\sigma = L/10$

- 数值参数
 - 基函数数量 $N_{basis} = 50$
 - 绘图点数 `num_x_points = 500`

```
In [7]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

# 参数定义
hbar = 1.0545718e-34
m_e = 9.1093837e-31
L = 1.0e-9

V0 = 5.0 * 1.602e-19
xc = L / 2
sigma = L / 10

N_basis = 50
num_x_points = 500

# 物理函数定义
def infinite_square_well_basis_function(x, n, L):
    if 0 <= x <= L:
        return np.sqrt(2.0 / L) * np.sin(n * np.pi * x / L)
    else:
        return 0.0

def infinite_square_well_kinetic_energy(n, L, hbar, m):
    return (n**2 * np.pi**2 * hbar**2) / (2.0 * m * L**2)

def perturbation_potential(x, V0, xc, sigma):
    return V0 * np.exp(-(x - xc)**2 / (2.0 * sigma**2))

def total_potential(x, V0, xc, sigma, L):
    if 0 <= x <= L:
        return perturbation_potential(x, V0, xc, sigma)
    else:
        return 1e10

# 哈密顿量矩阵
H_matrix = np.zeros((N_basis, N_basis))

for m in range(1, N_basis + 1):
    for n in range(1, N_basis + 1):
        if m == n:
            H_matrix[m-1, n-1] += infinite_square_well_kinetic_energy(n, L, hbar

            integrand = lambda x_val: (
                infinite_square_well_basis_function(x_val, m, L) *
                perturbation_potential(x_val, V0, xc, sigma) *
                infinite_square_well_basis_function(x_val, n, L)
            )

            integral_val, abserr = quad(integrand, 0, L)
            H_matrix[m-1, n-1] += integral_val
```

```

        # if m > n:
        #     H_matrix[m-1, n-1] = H_matrix[n-1, m-1]

# 矩阵对角化求解本征值和本征向量
eigenvalues, eigenvectors = np.linalg.eigh(H_matrix)

e_charge = 1.602176634e-19
eigenvalues_eV = eigenvalues / e_charge

print("前五个能量本征值(eV):")
for i in range(min(5, N_basis)):
    print(f"E_{i+1} = {eigenvalues_eV[i]:.6f} eV")

# 可视化
x_vals = np.linspace(0, L, num_x_points)

plt.figure(figsize=(12, 8))
plt.plot(x_vals * 1e9, [total_potential(x, V0, xc, sigma, L) / e_charge for x in
                        'k--', label='V(x) (eV)')

num_states_to_plot = min(5, N_basis)

for i in range(num_states_to_plot):
    current_coeffs = eigenvectors[:, i]

    # 选取了实波函数
    psi_x = np.zeros_like(x_vals, dtype=complex)
    for n in range(1, N_basis + 1):
        psi_x += current_coeffs[n-1] * np.array([infinite_square_well_basis_func

    psi_x = psi_x.real

    norm_factor = np.sqrt(np.trapz(np.abs(psi_x)**2, x_vals))
    if norm_factor > 1e-10:
        psi_x /= norm_factor

    prob_density = np.abs(psi_x)**2

    energy_offset_eV = eigenvalues_eV[i]

    # 因子，放大波函数，否则振幅过小
    plot_factor = L / 2
    plt.plot(x_vals * 1e9, psi_x * plot_factor + energy_offset_eV,
             label=f'$\psi_{i+1}(x)$')

    plt.plot(x_vals * 1e9, prob_density * plot_factor + energy_offset_eV,
             '--', label=f'$|\psi_{i+1}(x)|^2$', alpha=0.7)

    plt.axhline(y=energy_offset_eV, color='gray', linestyle=':',
                label=f'$E_{i+1}$ = {energy_offset_eV:.3f} eV' if i == 0 else "")

plt.title('Eigenfunctions and Probability Densities in a Perturbed Infinite Squa
plt.xlabel('Position x (nm)')
plt.ylabel('Energy (eV) / Wavefunction Amplitude (arbitrary units)')
plt.legend(loc='upper right', fontsize='small')
plt.grid(True, linestyle=':', alpha=0.6)
plt.xlim(0, L * 1e9)
plt.ylim(0, 10)
plt.show()

```

```

<>:99: SyntaxWarning: invalid escape sequence '\p'
<>:102: SyntaxWarning: invalid escape sequence '\p'
<>:99: SyntaxWarning: invalid escape sequence '\p'
<>:102: SyntaxWarning: invalid escape sequence '\p'
C:\Users\Voyager1\AppData\Local\Temp\ipykernel_2624\3157070335.py:99: SyntaxWarning: invalid escape sequence '\p'
    label=f'$\psi_{i+1}(x)$')
C:\Users\Voyager1\AppData\Local\Temp\ipykernel_2624\3157070335.py:102: SyntaxWarning: invalid escape sequence '\p'
    '--', label=f'$|\psi_{i+1}(x)|^2$', alpha=0.7)

```

前五个能量本征值(eV):

```

E_1 = 1.783630 eV
E_2 = 2.048854 eV
E_3 = 5.345546 eV
E_4 = 7.178405 eV
E_5 = 10.834582 eV

```

```

C:\Users\Voyager1\AppData\Local\Temp\ipykernel_2624\3157070335.py:88: DeprecationWarning: `trapz` is deprecated. Use `trapezoid` instead, or one of the numerical integration functions in `scipy.integrate`.
    norm_factor = np.sqrt(np.trapz(np.abs(psi_x)**2, x_vals))

```

